Day 3 - Transformers Interview Revision

Positional Encoding + Transformer Block

High-Yield Concepts

- Why Transformers need positional information
- Why self-attention alone does not know token order
- How positional encoding is added to token embeddings
- Sinusoidal positional encoding
- Why sine and cosine are both used
- Difference between positional encoding and positional embedding
- Transformer encoder block architecture
- Purpose of residual / skip connections
- Purpose of LayerNorm
- Purpose of the Feed-Forward Network (FFN)
- Why the FFN processes tokens independently
- Why the FFN expands into a larger hidden dimension
- Why nonlinear activation is required

Positional Encoding

1. Why do Transformers need positional encoding?

Transformers need positional encoding because self-attention does not inherently know the position or order of tokens in a sequence. For example, 'the cat jumped over the wall' and 'the wall jumped over the cat' contain the same words but have completely different meanings. Positional encoding adds information about each token's position.

2. Why can't self-attention alone understand the order of tokens?

Self-attention determines relationships between tokens using Q, K, and V, but the attention operation itself does not explicitly contain information saying which token is first, second, third, and so on. Positional information therefore needs to be added to the token representations.

3. How is positional encoding incorporated into the Transformer input?

Positional encoding is added element-wise to the token embeddings before the representation enters the Transformer block.

Token Embedding + Positional Encoding

$$X = E + PE$$

E = token embedding
PE = positional encoding
X = representation passed into the Transformer

Sinusoidal Positional Encoding

The original Transformer uses sine and cosine functions alternately across the embedding dimensions. Different dimensions use different frequencies, producing a structured representation of position.

PE(pos, 2i) = sin(pos / 10000^(2i / d))

PE(pos, 2i + 1) = cos(pos / 10000^(2i / d))

Why Use Both Sine and Cosine?

Sine is periodic, so different phases can produce the same sine value. For example, sin(30 degrees) = sin(150 degrees). Using sine and cosine together provides both components of the phase, making the positional representation more distinguishable.

sin(30 degrees) = sin(150 degrees) = 0.5

The pair (sin(theta), cos(theta)) provides more information about the phase than either function alone.

Positional Encoding vs Positional Embedding

Feature	Positional Encoding	Positional Embedding
Parameters	Fixed / deterministic	Learned parameters
Example	Sinusoidal functions	Learned lookup table
Training	Does not require learning	Updated during training
Representation	Generated from position	Learned vector for each position

Transformer Encoder Block

4. Walk through the architecture of a Transformer encoder block.

The input representation first passes through Multi-Head Self-Attention. A residual connection adds the original input, followed by LayerNorm. The result then passes through the Feed-Forward Network. Another residual connection and LayerNorm produce the output of the block.

Transformer Block Flow

X

↓

Multi-Head Self-Attention

↓

Residual Connection

↓

LayerNorm

↓

Feed-Forward Network

↓

Residual Connection

↓

LayerNorm

↓

Output

Residual / Skip Connections

5. Why do we use residual or skip connections?

Residual connections provide a shortcut path for information and gradients. For a transformation F(x) + x, the identity path has derivative 1, helping gradients propagate through deep networks and mitigating vanishing-gradient problems.

X1 = LayerNorm(X + MHSA(X))

X2 = LayerNorm(X1 + FFN(X1))

Layer Normalization

6. Why do we use LayerNorm?

LayerNorm normalizes activations across the feature dimensions of each individual token. This helps keep activation scales stable, making optimization and gradient flow more stable and reducing the risk of unstable activation or gradient behavior.

Important: LayerNorm does not provide nonlinearity. The activation function inside the FFN provides the nonlinearity.

LayerNorm vs BatchNorm

7. Why is LayerNorm preferred over BatchNorm in Transformers?

LayerNorm normalizes each token independently across its feature dimensions and does not depend on statistics from other examples in the batch. This makes it well suited to sequential data and variable sequence lengths.

Feed-Forward Network

8. What is the purpose of the Feed-Forward Network?

Self-attention captures contextual relationships between tokens. The FFN then applies a nonlinear transformation independently to each token, allowing the model to learn more complex representations.

9. Does the FFN allow tokens to interact with each other?

No. The FFN processes each token independently using the same weights. Token-to-token interaction occurs in the self-attention layer.

10. What is the difference between self-attention and the FFN?

Self-attention: allows tokens to interact and captures contextual relationships.
FFN: processes each token independently and applies a nonlinear transformation to its contextualized representation.

Why Is the Same FFN Used for Every Token?

11. Why is the same FFN applied independently to every token?

The transformation should be position-independent, so the same learned FFN can be applied to every token. Sharing the weights reduces the number of parameters and allows the same transformation to be used at every position.

FFN Dimension Expansion

12. Why does the Transformer use a larger hidden dimension inside the FFN?

The FFN expands the representation into a higher-dimensional space, giving the network more capacity to learn complex nonlinear transformations. It then projects the representation back to the model dimension so it can participate in the residual connection.

    d_model = 512 -> d_ff = 2048 -> d_model = 512

Feed-Forward Network Formula

13. What is the mathematical formula for the Transformer FFN?

    FFN(x) = W2 * ReLU(W1*x + b1) + b2

W1 expands the representation, the activation function introduces nonlinearity, and W2 projects the representation back to the model dimension.

The original Transformer used ReLU. Modern Transformers may use activations such as GELU or SwiGLU.

Why Is Nonlinearity Necessary?

14. Why can't we just use a single linear layer instead of the FFN?

Without a nonlinear activation, multiple linear or affine transformations can be reduced to another affine transformation. The nonlinear activation allows the network to learn complex nonlinear functions.

    W2(W1*x + b1) + b2 = W2*W1*x + (W2*b1 + b2)

Complete Transformer Encoder Flow

    Input Tokens
    ↓
    Token Embeddings
    ↓
    Add Positional Encoding
    ↓
    X
    ↓
    Multi-Head Self-Attention
    ↓
    Residual + LayerNorm

↓

Feed-Forward Network

↓

Residual + LayerNorm

↓

Output

The complete encoder block is then stacked repeatedly. Each block progressively refines the contextual representation.

Interview-Ready Summary

A Transformer first converts input tokens into embeddings and adds positional information. The representation then passes through Multi-Head Self-Attention, where tokens interact and contextual relationships are captured. A residual connection and LayerNorm follow. The result is then passed through a Feed-Forward Network, which independently applies a nonlinear transformation to each token. A second residual connection and LayerNorm produce the block output. This block is stacked multiple times.

Rapid-Fire Memory Sheet

Component	Purpose
Positional Encoding	Adds token position / ordering information.
Self-Attention	Allows tokens to interact and capture context.
Residual Connection	Provides a shortcut for information and gradient flow.
LayerNorm	Stabilizes activation scale and distribution.
FFN	Applies nonlinear transformations independently to each token.
FFN Activation	Provides nonlinear expressive power.
Stacked Blocks	Progressively refine token representations.

Core Flow to Memorize

> Tokens -> Embeddings + Positional Encoding -> MHSA -> Residual + LayerNorm -> FFN -> Residual + LayerNorm

Key distinction: Attention mixes information across tokens; the FFN transforms each token independently.

Key distinction: Residual connections help gradient flow; LayerNorm stabilizes activations; the FFN activation provides nonlinearity.